

Projet Apprentissage Statistique - Gabriel Stremsdoerfer et Denis Jaouen

Table of contents

1	Traitement des données	1
1.1	Importation des données et librairies	1
1.2	Discrétisation de Y	3
2	Statistiques descriptives	12
3	Création d'un jeu de donnée imputé avec le package missmda	20
4	Stacking de XGboost et RF	22
4.1	Préparation des données	22
4.2	Séparation apprentissage / test	22
5	Paramètres de RF	23
5.1	Paramètres de XGBoost	26
6	Stacking	30
6.1	Préparation des folds	30
6.2	Méta-modèle multinomial	33
6.3	RF et XGBoost finaux	33
6.4	Prédictions du stacking	34
7	Premiers résultats	35
7.1	Classes prédites	35
7.2	Matrices de confusion	35
7.3	Comparaison des accuracies	36
7.4	Matrices de confusion pondérées	36
7.5	Importance des variables	37
7.6	Résultats finaux à comparer	37

1 Traitement des données

1.1 Importation des données et librairies

```
library(mice)
```

```
Attaching package: 'mice'
```

```
The following object is masked from 'package:stats':
```

```
filter
```

```
The following objects are masked from 'package:base':
```

cbind, rbind

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks mice::filter(), stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
```

```
library(FactoMineR)
```

```
library(missMDA)
```

```
library(ranger)
```

```
library(tidyr)
```

```
library(caret)
```

Loading required package: lattice

Attaching package: 'caret'

The following object is masked from 'package:purrr':

lift

```
library(ggplot2)
```

```
library(readr)
```

```
library(purrr)
```

```
library(plotROC)
```

```
library(xgboost)
```

```
library(yardstick)
```

Attaching package: 'yardstick'

The following objects are masked from 'package:caret':

precision, recall, sensitivity, specificity

The following object is masked from 'package:readr':

spec

```
library(forcats)
```

```
library(SHAPforxgboost)
```

Registered S3 method overwritten by 'car':

method from

na.action.merMod lme4

```
library(questionr)
```

```
library(nnet)
```

```
library(purrr)
```

```
setwd(getwd())
escap = read.csv("ESCAP.csv", sep = ';')
summary(escap)
```

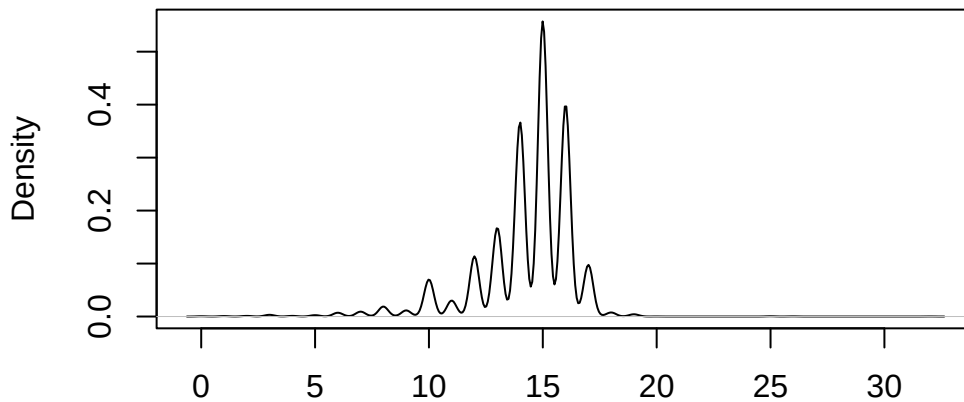
A01	Q03	Q04	Q04A		
Min. : 1	Min. :1.000	Min. :1.000	Min. :1.000		
1st Qu.:10795	1st Qu.:1.000	1st Qu.:1.000	1st Qu.:1.000		
Median :22544	Median :1.000	Median :1.000	Median :1.000		
Mean :22534	Mean :1.494	Mean :1.042	Mean :1.099		
3rd Qu.:34228	3rd Qu.:2.000	3rd Qu.:1.000	3rd Qu.:1.000		
Max. :45562	Max. :2.000	Max. :2.000	Max. :3.000		
	NAs :23	NAs :1583	NAs :593		
Q04B	Q05	Q06A	Q06B		
Min. :1.000	Min. :1.000	Min. :1.000	Min. :1.000		
1st Qu.:1.000	1st Qu.:1.000	1st Qu.:1.000	1st Qu.:1.000		
Median :2.000	Median :1.000	Median :1.000	Median :1.000		
Mean :2.177	Mean :1.302	Mean :1.154	Mean :1.288		
3rd Qu.:2.000	3rd Qu.:2.000	3rd Qu.:1.000	3rd Qu.:2.000		
Max. :4.000	Max. :2.000	Max. :3.000	Max. :3.000		
NAs :12777	NAs :40	NAs :34	NAs :40		
Q08	Q08C	Q09A1	Q09B1	Q10A1	
Min. :1.000	Min. :1.00	Min. :1.000	Min. :1.000	Min. :1.000	
1st Qu.:1.000	1st Qu.:1.00	1st Qu.:1.000	1st Qu.:1.000	1st Qu.:4.000	
Median :1.000	Median :1.00	Median :1.000	Median :1.000	Median :6.000	
Mean :1.145	Mean :1.47	Mean :1.601	Mean :1.494	Mean :5.219	
3rd Qu.:1.000	3rd Qu.:2.00	3rd Qu.:1.000	3rd Qu.:1.000	3rd Qu.:7.000	
Max. :5.000	Max. :5.00	Max. :7.000	Max. :7.000	Max. :9.000	
NAs :95	NAs :81	NAs :402	NAs :256	NAs :1536	
Q10B1	Q19A	B08A	B08B		
Min. :1.000	Min. : 0.0	Min. :1.000	Min. :1.000		
1st Qu.:5.000	1st Qu.:14.0	1st Qu.:2.000	1st Qu.:1.000		
Median :6.000	Median :15.0	Median :3.000	Median :2.000		
Mean :5.636	Mean :14.3	Mean :2.944	Mean :2.474		
3rd Qu.:6.000	3rd Qu.:16.0	3rd Qu.:4.000	3rd Qu.:4.000		
Max. :9.000	Max. :32.0	Max. :5.000	Max. :5.000		
NAs :1475	NAs :2393	NAs :1400	NAs :1050		
pm17B					
Length :13314					
N.unique : 1305					
N.blank : 0					
Min.nchar: 8					
Max.nchar: 11					

1.2 Discretisation de Y

```
escap$Y <- escap$Q19A
escap = escap[,-15]

plot(density(escap$Y, na.rm = TRUE))
```

density(x = escap\$Y, na.rm = TRUE)



N = 10921 Bandwidth = 0.2092

```
q <- quantile(
  escap$Y,
  probs = c(0.20, 0.40, 0.60, 0.80),
  na.rm = TRUE
)

escap$Y <- cut(
  escap$Y,
  breaks = c(-Inf, q, Inf),
  labels = c("13-", "14", "15", "16", "17+"),
  include.lowest = TRUE
)

escap$Y <- as.factor(escap$Y)

table(escap$Y)
```

```
13-  14  15  16  17+
2551 2150 3244 2331  645
```

La distribution de Y a des queues de distribution très aplaties, ce qui peut poser problème en l'état en introduisant des classes rares. Pour maintenir des effectifs équilibrés sans perdre trop d'information, on choisit alors d'utiliser les quintiles pour discrétiser la variable. Les modalités finales de Y sont donc : 13- (2551 individus), 14 (2150 individus), 15 (3244 individus), 16 (2331 individus) et 17+ (645 individus)

```
#Q03 : sexe
indiv_nr_Q03 <- escap[which(is.na(escap$Q03)),1]

#Q04
indiv_nr_Q04 <- escap[which(is.na(escap$Q04)),1]

#Q04 A et B: situation scolaire (A) et pro (B)
# Hypothèse : la situation scolaire et professionnelle s'excluent : NA = non concerné
# individus NR pour A et B
```

```

indiv_nr_Q04AB <- escap |>
  filter(is.na(Q04A) & is.na(Q04B)) |>
  select(A01)

#Q0A : individus répondants et non répondants exceptés individus NR pour A et B
indiv_nr_Q04A <- escap |>
  filter(!(A01 %in% indiv_nr_Q04AB$A01)) |>
  filter(is.na(Q04A)) |>
  dplyr::select(A01)

indiv_r_Q04A <- escap |>
  filter(!(is.na(Q04A))) |>
  dplyr::select(A01)

#Q0B : individus répondants et non répondants exceptés individus NR pour A et B
indiv_nr_Q04B <- escap |>
  filter(!(A01 %in% indiv_nr_Q04AB$A01)) |>
  filter(is.na(Q04B)) |>
  dplyr::select(A01)

indiv_r_Q04B <- escap |>
  filter(!(is.na(Q04B))) |>
  dplyr::select(A01)

#individus hors intersection entre répondre à Q04B et ne pas répondre à Q04A
diff_r_Q0B_nr_Q0A <- setdiff(
  indiv_nr_Q04A,
  indiv_r_Q04B
) #0

#individus hors intersection entre répondre à Q04A et ne pas répondre à Q04B
diff_r_Q0A_nr_Q0B <- setdiff(
  indiv_nr_Q04B,
  indiv_r_Q04A
) #0

#on en conclut que ormis les 76 : tous non concernés
pds_indiv_nr_Q04AB <- escap |>
  filter(A01 %in% indiv_nr_Q04AB$A01) |>
  select(pm17B)

pds_indiv_nr_Q04AB <- pds_indiv_nr_Q04AB$pm17B

#On impute une nouvelle modalité : 0 = "Non concerné" pour Q04A et Q04B

#Q04A
escap <- escap |>
  mutate(
    Q04A = ifelse(
      A01 %in% indiv_nr_Q04A$A01,
      0,
      Q04A
    )
  )

#Q04B

```

```

escap <- escap |>
  mutate(
    Q04B = ifelse(
      A01 %in% indiv_nr_Q04B$A01,
      0,
      Q04B
    )
  )

indiv_nr_Q04AB <- indiv_nr_Q04AB$A01

#Q04
# est-ce que le volume de NR peut être réduit avec info de Q04A et Q04B ?

indiv_r_Q04AB <- escap |>
  filter(!(is.na(Q04A)) & !(is.na(Q04B))) |>
  select(A01)

indiv_r_Q04AB <- indiv_r_Q04AB$A01

length(setdiff(indiv_nr_Q04, indiv_r_Q04AB))

```

```
[1] 19
```

```

# 19 indiv NR de Q04 ne sont pas parmi les indiv qui ont répondu à Q04A ou Q04B

# on impute la réponse en fonction des réponses cochées dans Q04A et Q04B
escap <- escap |>
  mutate(
    Q04 = ifelse(is.na(Q04A), NA, 1)
  ) |>
  mutate(
    Q04 = ifelse(
      !(Q04B %in% c(0,NA)),
      2,
      Q04
    )
  )

#ici on a une hypothèse : travailler (ou rechercher) = arrêter ses études
#néanmoins on a vu que les non réponses dans Q04A et Q04B étaient vraisemblablement des
#réponses de non concerné : donc pour nos individus cette hypothèse est crédible.

#Q05 : redoublement
any(is.na(escap$Q05))#TRUE

```

```
[1] TRUE
```

```

indiv_nr_Q05 <- escap |>
  filter(is.na(Q05)) |>
  select(A01)

indiv_nr_Q05 <- indiv_nr_Q05$A01

setdiff(
  indiv_nr_Q05,
  indiv_nr_Q04AB
)

```

```
) #36 sur 40 sont pas parmi les non répondants de A et B
```

```
[1] 44056 25021 23876 40027 22499 30068 36015 22442 21725 20859 5457 14473
[13] 22679 6840 36656 13624 24448 33544 27627 35446 33384 9751 28791 23735
[25] 31439 14702 5593 17216 4751 34189 2561 15369 26649 1575 22524 35792
```

Ici, on s'est intéressé aux variables Q04, Q04A et Q04B. C'est à dire les variables traitant de la situation scolaire et/ou professionnelle. Avec quelques manipulations, on a pu réduire la non-réponse conditionnelle présente dans ces variables. C'est à dire que dans la grande majorité des cas, un répondant qui travaille ne fait pas d'étude et inversement.

```
#Q06A et B
```

```
any(is.na(escap$Q06A))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q06A <- escap |>
  filter(is.na(Q06A)) |>
  select(A01)
```

```
indiv_nr_Q06A <- indiv_nr_Q06A$A01
```

```
any(is.na(escap$Q06B))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q06B <- escap |>
  filter(is.na(Q06B)) |>
  select(A01)
```

```
indiv_nr_Q06B <- indiv_nr_Q06B$A01
```

```
#intersections entre non répondants ?
```

```
setdiff(indiv_nr_Q06B, indiv_nr_Q06A)
```

```
[1] 41866 43295 5446 1227 24573 37430 44372 27802 39272 8687 6546 21045
[13] 568 6840 5619 4027 11761 32849 32858 11989 17696 36669 34033 18718
[25] 31439 21474 13329 32940 14563 26711 41099 43047 4598 12877
```

```
setdiff(indiv_nr_Q06A, indiv_nr_Q06B)
```

```
[1] 41453 2404 33254 35373 3985 44429 12427 17375 31562 23165 12046 8536
[13] 15563 28009 7865 21072 9067 19770 23054 29854 29141 636 36871 42539
[25] 34968 10797 20892 38571
```

```
#globalement, peu d'individus n'ont pas répondu aux deux questions
```

```
#Q08
```

```
any(is.na(escap$Q08))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q08 <- escap |>
  filter(is.na(Q08)) |>
  select(A01)
```

```
indiv_nr_Q08 <- indiv_nr_Q08$A01
```

```
#Q08C
```

```

indiv_nr_Q08C <- escap |>
  filter(is.na(Q08C)) |>
  select(A01)

indiv_nr_Q08C <- indiv_nr_Q08C$A01

#individus qui ont pas connus leur parent pour X raison => réduire NR des réponses sur parents

indiv_sans_parents <- escap |>
  filter(Q08C %in% c(3,4)) |>
  select(A01)

indiv_sans_parents <- indiv_sans_parents$A01 #538

indiv_sans_pere <- escap |>
  filter(Q09A1 == 7) |>
  select(A01)

indiv_sans_pere <- indiv_sans_pere$A01 #554 individus sans père

indiv_sans_mere <- escap |>
  filter(Q09B1 == 7) |>
  select(A01)

indiv_sans_mere <- indiv_sans_mere$A01 #157 individus sans mere

#Q09A1
indiv_nr_Q09A1 <- escap |>
  filter(is.na(Q09A1)) |>
  select(A01)

indiv_nr_Q09A1 <- indiv_nr_Q09A1$A01 #402 non répondants

#indiv qui ne savent pas :
indiv_nsp_pere <- escap |>
  filter(Q09A1 == 6) |>
  select(A01)

indiv_nsp_pere <- indiv_nsp_pere$A01

#Q09B1
indiv_nr_Q09B1 <- escap |>
  filter(is.na(Q09B1)) |>
  select(A01)

indiv_nr_Q09B1 <- indiv_nr_Q09B1$A01

#indiv qui ne savent pas :
indiv_nsp_mere <- escap |>
  filter(Q09B1 == 6) |>
  select(A01)

indiv_nsp_mere <- indiv_nsp_mere$A01

```

```

#Q10A1
escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_sans_pere,
      9,
      Q10A1
    )
  )

escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_sans_parents,
      9,
      Q10A1
    )
  )

#nouvelle modalité : ne sait pas
escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_nsp_pere,
      0,
      Q10A1
    )
  )

indiv_nr_Q10A1 <- escap |>
  filter(is.na(Q10A1)) |>
  select(A01)

indiv_nr_Q10A1 <- indiv_nr_Q10A1$A01

escap$Q10A1 <- as.factor(escap$Q10A1)


#Q10B1
escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_sans_mere,
      9,
      Q10B1
    )
  )

escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_sans_parents,
      9,
      Q10B1
    )
  )

```

```

    )
  )

#nouvelle modalité : ne sait pas
escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_nsp_mere,
      0,
      Q10B1
    )
  )

indiv_nr_Q10B1 <- escap |>
  filter(is.na(Q10B1)) |>
  select(A01)

indiv_nr_Q10B1 <- indiv_nr_Q10B1$A01

escap$Q10B1 <- as.factor(escap$Q10B1)

#B08A
any(is.na(escap$B08A))#TRUE

```

```
[1] TRUE
```

```

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01 #1400 non répondants

#ajout d'une nouvelle modalité : 0 = non concerné
escap <- escap |>
  mutate(
    B08A = ifelse(
      A01 %in% indiv_sans_pere,
      0,
      B08A
    )
  )

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01

escap <- escap |>
  mutate(
    B08A = ifelse(
      A01 %in% indiv_sans_parents,
      0,
      B08A
    )
  )

```

```

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01

escap$B08A <- as.factor(escap$B08A)

#B08B
any(is.na(escap$B08B))#TRUE

```

[1] TRUE

```

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01 #1050 non répondants

#on ajoute une modalité: 0 = non concerné
escap <- escap |>
  mutate(
    B08B = ifelse(
      A01 %in% indiv_sans_mere,
      0,
      B08B
    )
  )

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01

escap <- escap |>
  mutate(
    B08B = ifelse(
      A01 %in% indiv_sans_parents,
      0,
      B08B
    )
  )

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01

escap$B08B <- as.factor(escap$B08B)

```

En observant les différentes modalités de certaines questions, on a à nouveau pu réduire les non-réponses dans des variables liées. Notamment, lorsqu'un répondant déclare sa mère comme décédée ou perdue de vue, il ne peut logiquement pas déclarer sa consommation d'alcool.

2 Statistiques descriptives

tests d'indépendance du Khi2

```
# 1. Liste des variables explicatives à tester face à Y
vars <- c(
  "Q03", "Q04", "Q04A", "Q04B", "Q05",
  "Q06A", "Q06B", "Q08", "Q08C", "Q09A1",
  "Q09B1", "Q10A1", "Q10B1", "B08A", "B08B"
)

# 2. Exécution des tests du Chi2 et extraction des statistiques
chi2_results <- map_dfr(vars, function(v) {
  # Récupération du test du Chi2 (ignore silencieusement les avertissements d'effectifs faibles)
  test <- suppressWarnings(chisq.test(escap$Y, escap[[v]]))

  tibble(
    Variable = v,
    `Statistique (Chi2)` = round(unname(test$statistic), 2),
    `Degrés de liberté` = unname(test$parameter),
    `p-value` = test$p.value,
    Significatif = ifelse(test$p.value < 0.05, "Oui", "Non")
  )
})

# 3. Formater la p-value pour un affichage propre (ex: "< 0.001")
chi2_results <- chi2_results %>%
  mutate(
    `p-value` = ifelse(`p-value` < 0.001, "< 0.001", sprintf("%.4f", `p-value`))
  )

# Affichage du tableau dans la sortie de la cellule
chi2_results
```

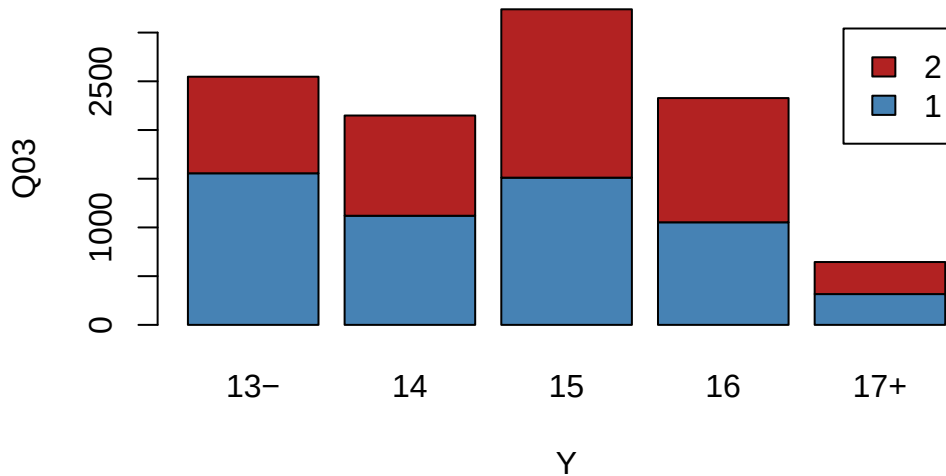
```
# A tibble: 15 x 5
  Variable `Statistique (Chi2)` `Degrés de liberté` `p-value` Significatif
  <chr>          <dbl>             <int> <chr>      <chr>
1 Q03             161.               4 < 0.001   Oui
2 Q04             21.5               4 < 0.001   Oui
3 Q04A            35.7              12 < 0.001   Oui
4 Q04B            30.6              16 0.0154   Oui
5 Q05            147.               4 < 0.001   Oui
6 Q06A            42.2               8 < 0.001   Oui
7 Q06B            55.9               8 < 0.001   Oui
8 Q08             22.5              16 0.1271   Non
9 Q08C            14.8              16 0.5357   Non
10 Q09A1           34.3              24 0.0796   Non
11 Q09B1           79.4              24 < 0.001   Oui
12 Q10A1          163.              36 < 0.001   Oui
13 Q10B1          128.              36 < 0.001   Oui
14 B08A           253.              20 < 0.001   Oui
15 B08B           238.              20 < 0.001   Oui
```

A priori, Y ne semble pas indépendantes de chacun des prédicteurs pris à un un.

```
#Barplot
#Q03 : sexe
tab <- table(escap[[2]], escap[[18]])
```

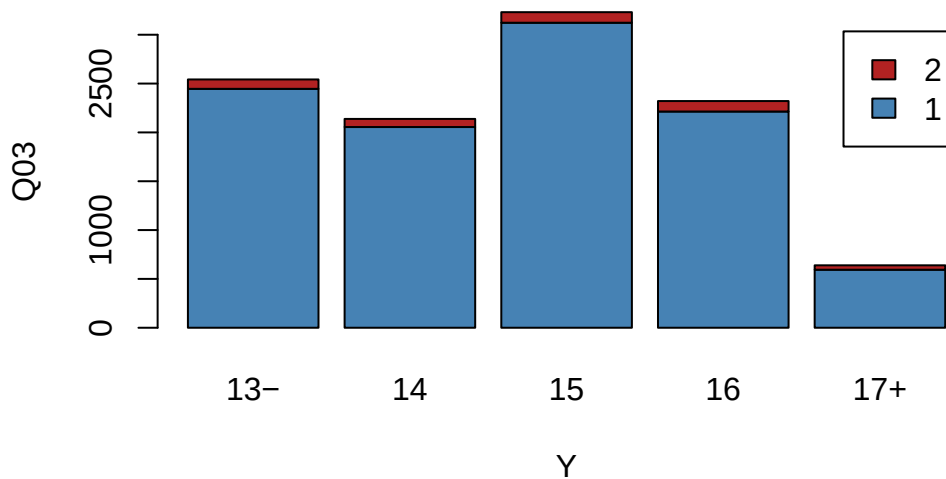
```
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition du sexe selon l'âge de
```

Répartition du sexe selon l'âge de première consommation



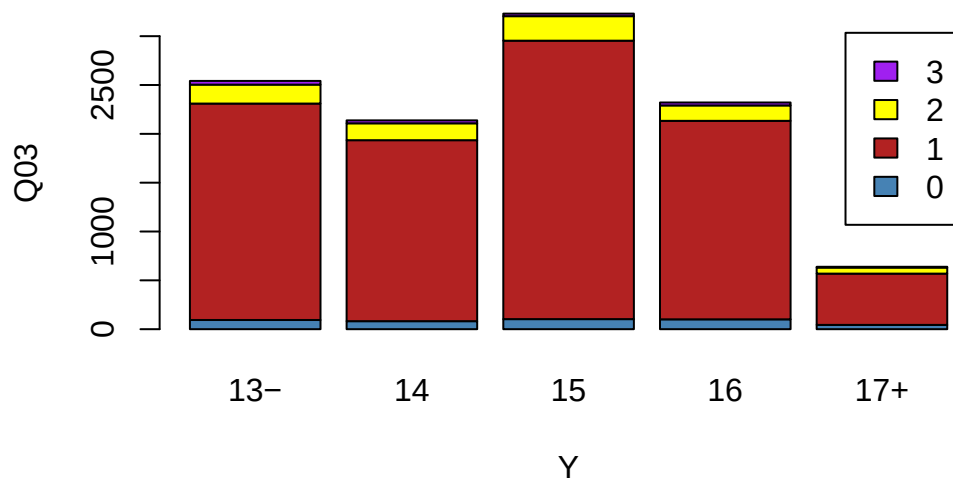
```
#Q04 : Situation
tab <- table(escap[[3]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition de la situation personnelle
```

Répartition de la situation personnelle selon l'âge de première consommation



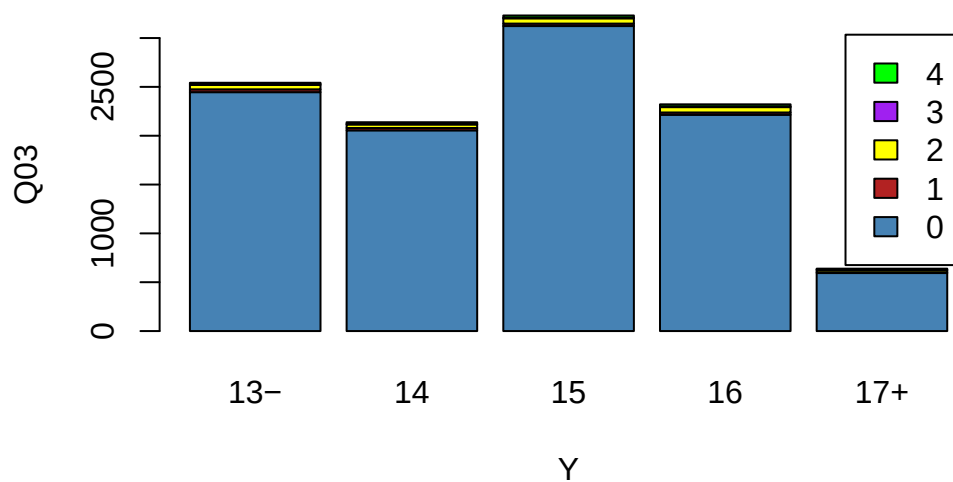
```
#Q04A : Situation scolaire
tab <- table(escap[[4]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple"), main = "Répartition de la
```

de la situation vis-à-vis des études selon l'âge de première



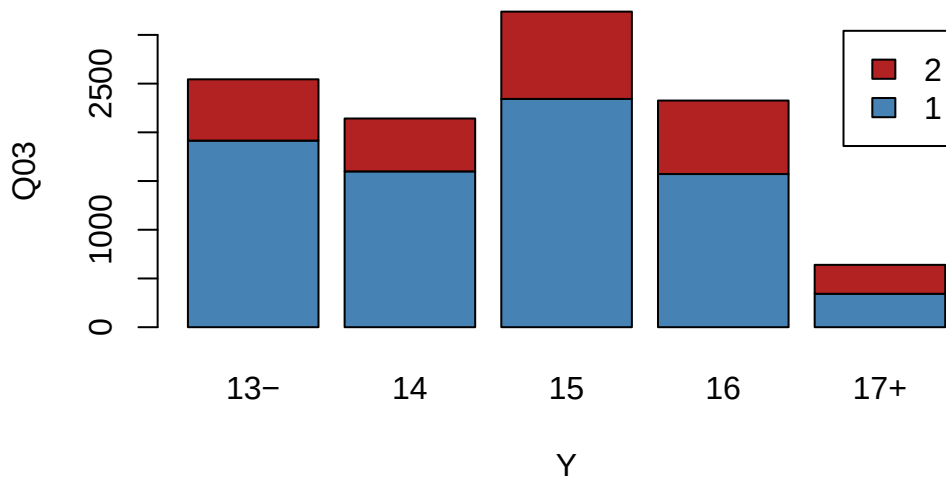
```
#Q04B : Situation pro
tab <- table(escap[[5]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "green"), main = "Répart
```

partition de la situation pro selon l'âge de première consom



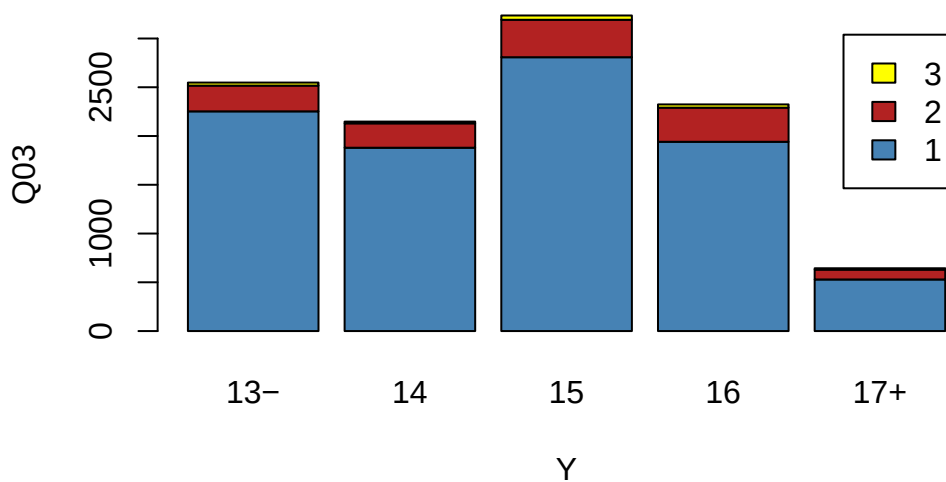
```
#Q05 : redoublement
tab <- table(escap[[6]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition du redoublement selon l
```

Répartition du redoublement selon l'âge de première consommation



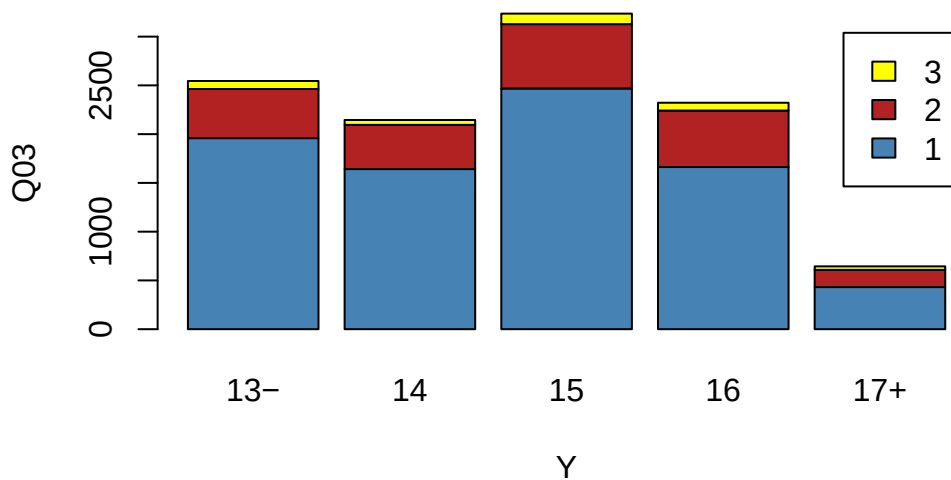
```
#Q06A
tab <- table(escap[[7]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow"), main = "Répartition du niveau de difficulté à lire selon l'âge de première consommation")
```

Répartition du niveau de difficulté à lire selon l'âge de première consommation



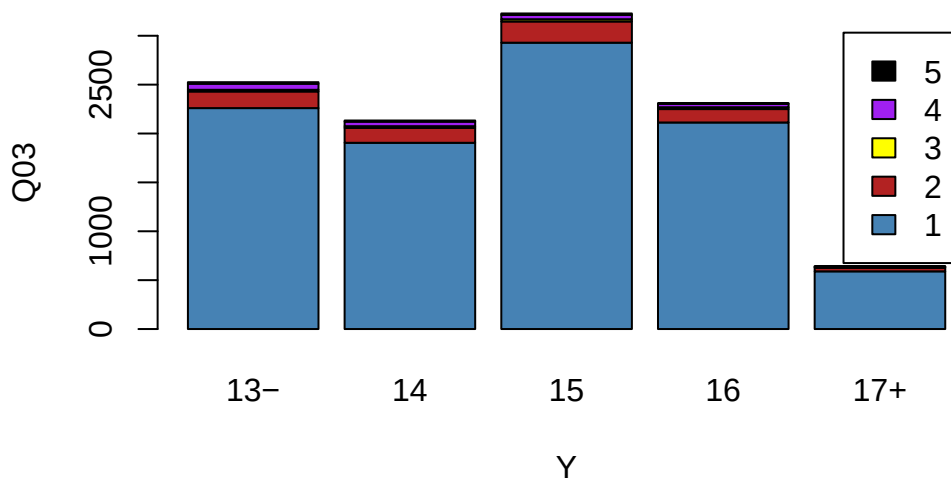
```
#Q06B
tab <- table(escap[[8]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow"), main = "Répartition du niveau de difficulté à lire selon l'âge de première consommation")
```

on du niveau de difficulté à écrire selon l'âge de première cc



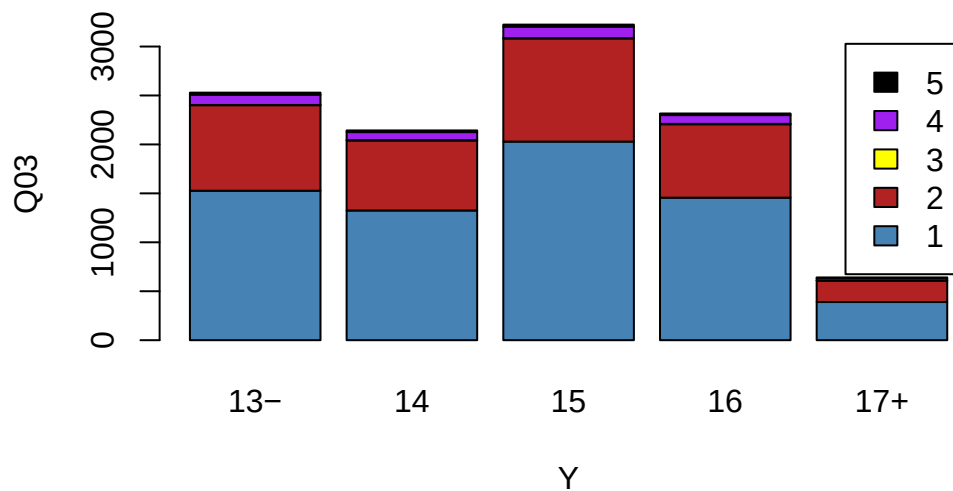
```
#Q08
tab <- table(escap[[9]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black"), main = "Répart
```

Répartition du lieu de vie selon l'âge de première consommation



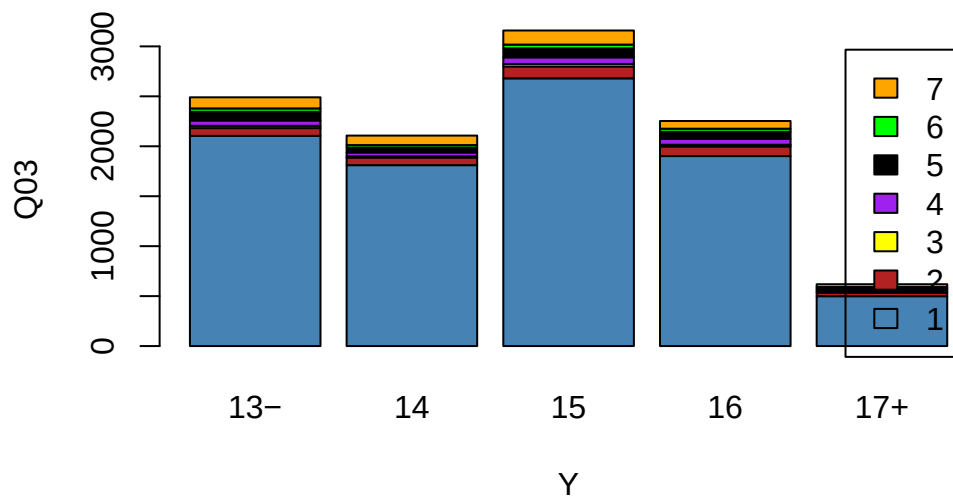
```
#Q08C
tab <- table(escap[[10]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black"), main = "Répart
```

ion de la configuration parentale selon l'âge de première co



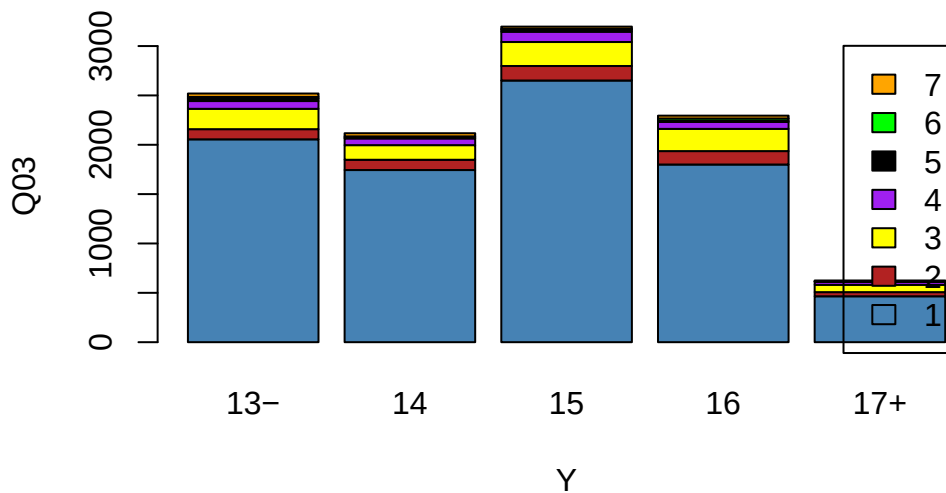
```
#Q09A1
tab <- table(escap[[11]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

rtition de la situation du père selon l'âge de première conso



```
#Q09B1
tab <- table(escap[[12]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

partition de la situation de la mère selon l'âge de première cons

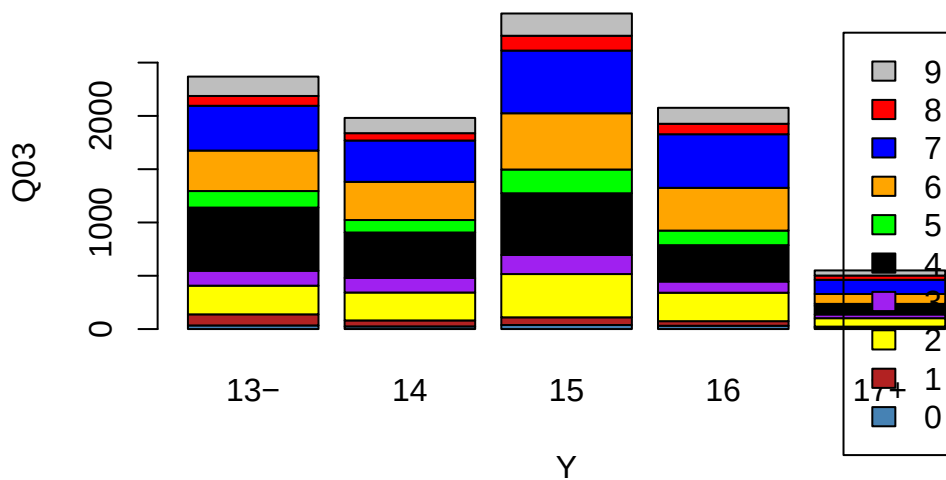


```
#Q10A1
```

```
tab <- table(escap[[13]], escap[[18]])
```

```
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

partition de la PCS du père selon l'âge de première consom

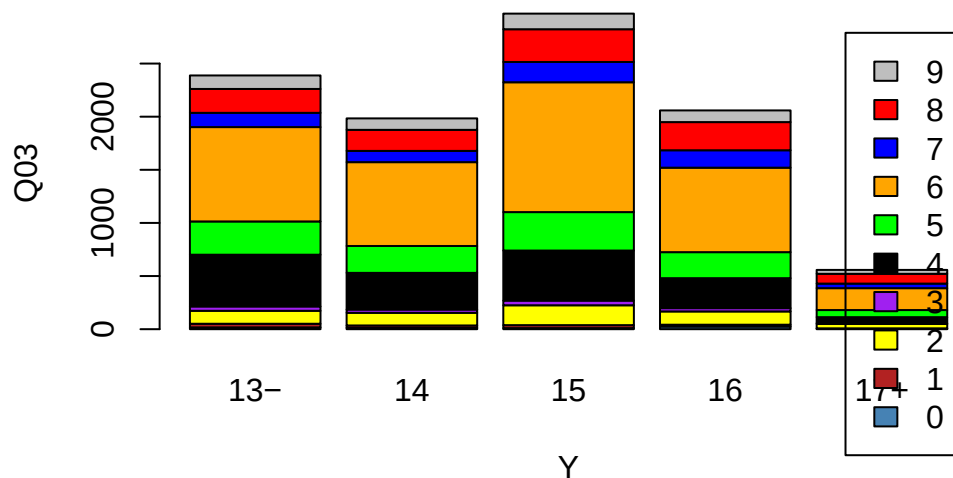


```
#Q10B1
```

```
tab <- table(escap[[14]], escap[[18]])
```

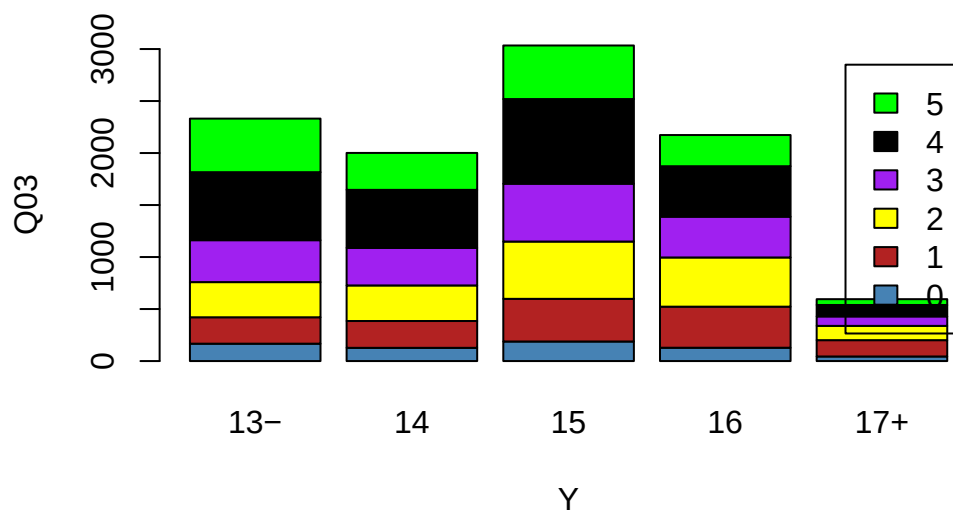
```
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

artition de la PCS de la mère selon l'âge de première consor



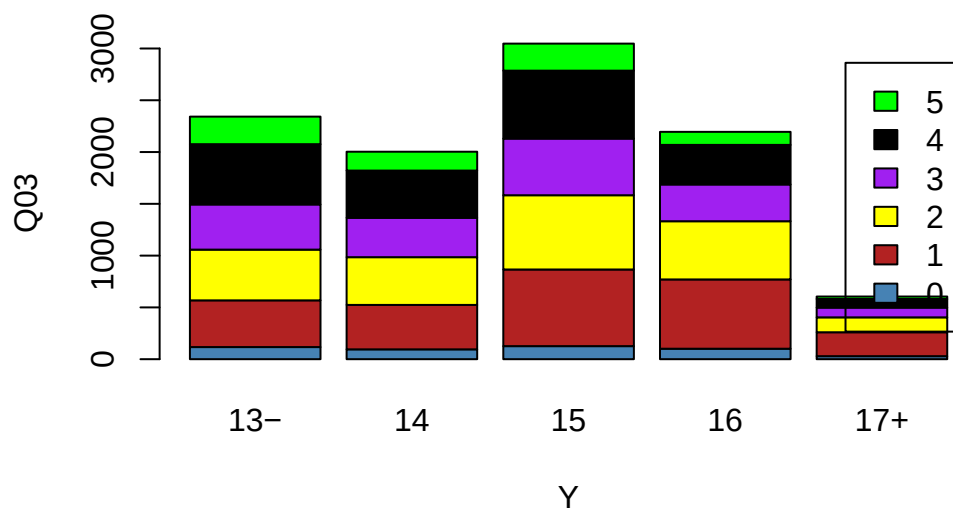
```
#B08A
tab <- table(escap[[15]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green"), main :
```

ion de la consommation du père selon l'âge de première coi



```
#B08B
tab <- table(escap[[16]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green"), main :
```

on de la consommation de la mère selon l'âge de première c



L'inspection des graphiques en barre croisés ne soulève pas de lien évident entre chaque prédicteur et Y. Dans chacune des modalités de Y, les modalités des prédicteurs semblent se transposer dans des proportions relativement similaires. On ne peut donc pas tirer de premières conclusions.

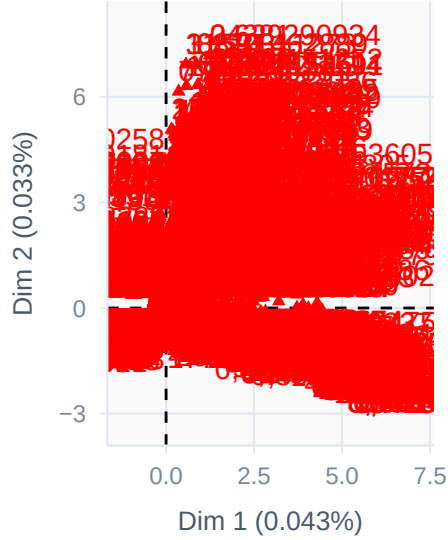
3 Création d'un jeu de donnée imputé avec le package missmda

Pour pouvoir mener nos analyser et traiter la non réponse partielle dans nos individus, nous choisissons d'imputer les données manquantes, tant dans les prédicteurs que dans Y. Nous utilisons le package missmda.

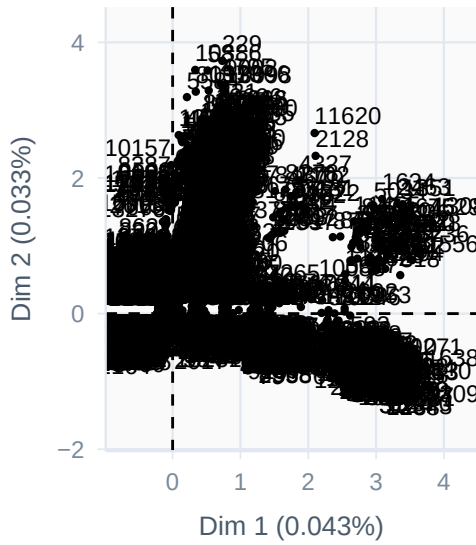
```
df_MCA <- escap |> select(-c(pm17B,A01))
df_MCA <- df_MCA |> mutate(across(everything(), as.factor))
rownames(df_MCA) <- escap$A01

res <- MCA(escap)
```

MCA factor map



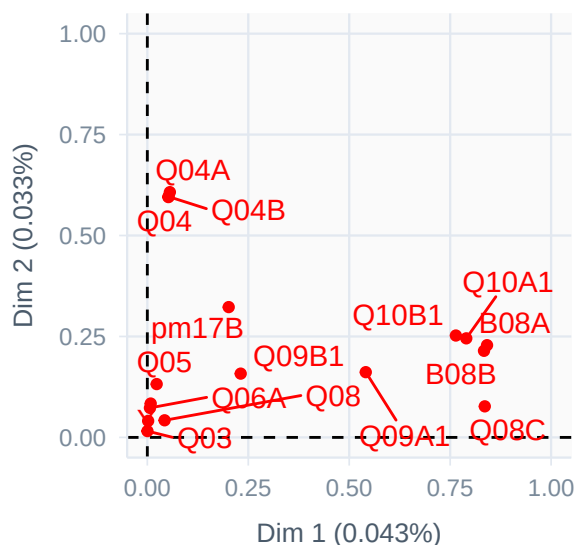
MCA factor map



```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_point()`).
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_text_repel()`).
```

Variables representation



```
# nb_comp <- estim_ncpMCA(df_MCA, ncp.max = 5, method.cv = "Kfold", nbsim = 20)
# nb_comp = 5
data_complete <- imputeMCA(df_MCA, 5)
data_complete <- data_complete$completeObs

write.csv(data_complete,
          file = "Impute_missMDA_gab.csv",
          row.names = FALSE)
```

4 Stacking de XGboost et RF

4.1 Préparation des données

```
data_rf <- read.csv2(
  "Impute_missMDA_gab.csv",
  sep = ",",
  stringsAsFactors = FALSE
)
data_rf <- data_rf |> mutate(across(1:16, as.factor))

poids_texte <- escap$pm17B

# 2. On remplace la virgule par un point avec gsub()
poids_propre <- gsub(",", ".", poids_texte)

# 3. On convertit en numeric et on l'assigne à data_rf
data_rf$pm17B <- as.numeric(poids_propre)
```

4.2 Séparation apprentissage / test

On utilise un seul découpage apprentissage/test pour les trois modèles afin que leurs performances soient comparables.

```
set.seed(42)
```

```

train_index <- createDataPartition(
  data_rf$Y,
  p = 0.70,
  list = FALSE
)

train_data <- data_rf[
  train_index,
  ,
  drop = FALSE
]

test_data <- data_rf[
  -train_index,
  ,
  drop = FALSE
]

classes <- levels(train_data$Y)
K <- length(classes)

# 5 folds communs à RF et XGBoost
set.seed(42)

folds <- createFolds(
  train_data$Y,
  k = 5,
  list = TRUE,
  returnTrain = FALSE
)

```

5 Paramètres de RF

```

# Sécurisation préalable des types
train_data$Y <- as.factor(as.character(train_data$Y))
rf_features <- setdiff(names(train_data), c("Y", "pm17B"))

# Variables explicatives
rf_features <- setdiff(
  names(train_data),
  c("Y", "pm17B")
)

# Hyperparamètres testés
rf_grid <- expand.grid(
  mtry = 1:7,
  num.trees = seq(50, 500, by = 50),
  min.node.size = c(1, 5, 10)
)

#####
# TUNING RANDOM FOREST PAR VALIDATION CROISEE
#####
rf_results <- rf_grid %>%

```

```

mutate(
  cv_error = pmap_dbl(
    list(mtry, num.trees, min.node.size),
    function(m, ntree, node) {

      fold_errors <- map_dbl(
        folds,
        function(valid_index) {

          train_fold <- train_data[-valid_index, , drop = FALSE]
          valid_fold <- train_data[valid_index, , drop = FALSE]

          # Entraînement avec ranger
          model <- ranger(
            formula      = Y ~ . - pm17B,
            data          = train_fold,
            mtry          = m,
            num.trees     = ntree,
            min.node.size = node,
            case.weights  = train_fold$pm17B,
            probability   = FALSE
          )

          # Prédiction sur le pli de validation
          pred <- predict(
            model,
            data = valid_fold[, rf_features, drop = FALSE]
          )$predictions

          # Calcul du taux d'erreur pondéré par les poids de sondage
          weighted.mean(pred != valid_fold$Y, w = valid_fold$pm17B)
        }
      )

      # Moyenne des erreurs sur l'ensemble des plis
      mean(fold_errors)
    }
  )
)

```

```

# Meilleur modèle
best_rf <- rf_results %>%
  slice_min(
    cv_error,
    n = 1,
    with_ties = FALSE
  )

rf_mtry <- best_rf$mtry #3
rf_ntree <- best_rf$num.trees #500
rf_node <- best_rf$min.node.size #10
#erreur CV = 0.5821
cat(
  "Meilleur modèle : mtry =",
  rf_mtry,
  ", ntree =",

```

```

rf_ntree,
", min.node.size =",
rf_node,
", erreur CV =",
round(best_rf$cv_error, 4),
"\n"
)

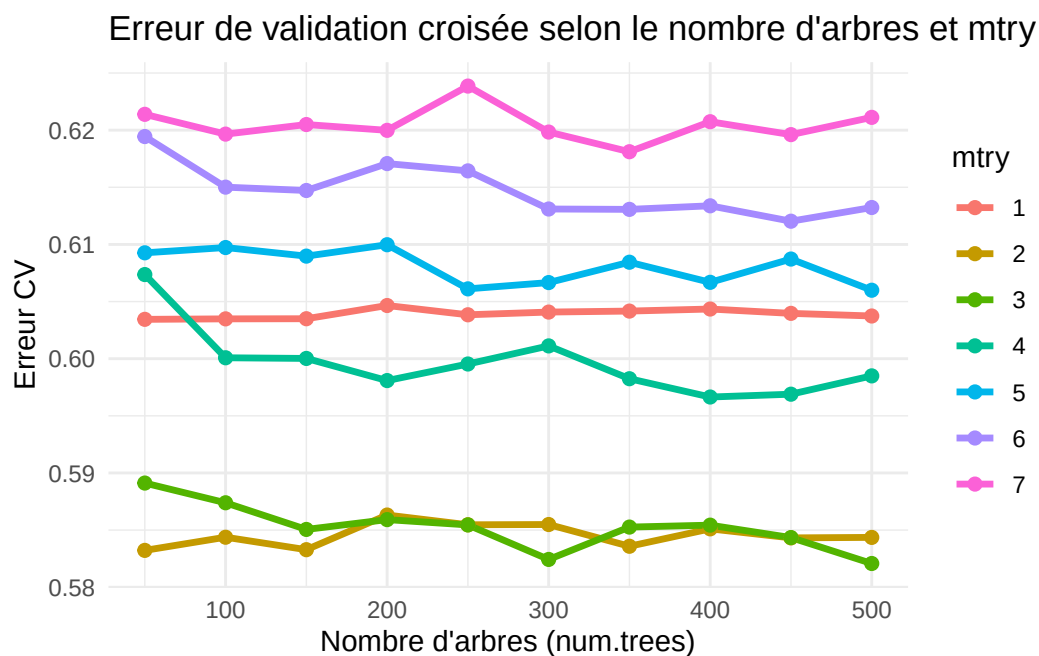
```

Meilleur modèle : mtry = 3 , ntree = 500 , min.node.size = 10 , erreur CV = 0.5821

```

# Visualisation de l'erreur CV
ggplot(
  rf_results %>%
    filter(
      min.node.size == rf_node
    ),
  aes(
    x = num.trees,
    y = cv_error,
    color = as.factor(mtry)
  )
) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  labs(
    title = "Erreur de validation croisée selon le nombre d'arbres et mtry",
    x = "Nombre d'arbres (num.trees)",
    y = "Erreur CV",
    color = "mtry"
  ) +
  theme_minimal()

```



```
rf_mtry # 3
```

```
[1] 3
```

```
rf_ntree #500
```

```
[1] 500
```

```
rf_node #10
```

```
[1] 10
```

Voici donc les paramètres optimaux pour notre Random Forest.

5.1 Paramètres de XGBoost

```
#####
# TUNING XGBOOST PAR VALIDATION CROISEE
#####

# Transformation des variables qualitatives en variables numériques
x_train <- model.matrix(
  Y ~ . - pm17B - 1,
  data = train_data
)

x_test <- model.matrix(
  Y ~ . - pm17B - 1,
  data = test_data
)

# Même ordre de colonnes pour train et test
x_test <- x_test[
  ,
  colnames(x_train),
  drop = FALSE
]

# Classes numériques 0, ..., K-1
y_train <- as.numeric(
  train_data$Y
) - 1

y_test <- as.numeric(
  factor(
    test_data$Y,
    levels = classes
  )
) - 1

# Poids
w_train <- train_data$pm17B
w_test <- test_data$pm17B

#####
# HYPERPARAMETRES TESTES
#####

xgb_grid <- expand.grid(
  max_depth = c(2, 3, 4, 5, 6),
  nrounds = c(50, 100, 150, 200, 300),
```

```

eta = c(0.01, 0.05, 0.10, 0.20)
)

#####
# VALIDATION CROISEE
#####

xgb_results <- xgb_grid %>%
  mutate(
    cv_error = pmap_dbl(
      list(
        max_depth,
        nrounds,
        eta
      ),
      function(depth, rounds, learning_rate) {

        fold_errors <- map_dbl(
          folds,
          function(valid_index) {

            train_index_fold <- setdiff(
              seq_len(nrow(train_data)),
              valid_index
            )

            dtrain <- xgb.DMatrix(
              x_train[
                train_index_fold,
              ],
              drop = FALSE
            ),
            label = y_train[
              train_index_fold
            ],
            weight = w_train[
              train_index_fold
            ]
          )

            dvalid <- xgb.DMatrix(
              x_train[
                valid_index,
              ],
              drop = FALSE
            ),
            label = y_train[
              valid_index
            ]
          )

            model <- xgb.train(
              params = list(
                objective = "multi:softprob",
                num_class = K,
                max_depth = depth,

```

```

        eta = learning_rate,
        eval_metric = "merror"
    ),
    data = dtrain,
    nrounds = rounds,
    verbose = 0
)

# Probabilités prédites
pred_prob <- predict(
  model,
  dvalid
)

pred_prob <- matrix(
  pred_prob,
  ncol = K,
  byrow = TRUE
)

# Classe prédite
pred_class <- max.col(
  pred_prob
) - 1

# Erreur de classification
mean(
  pred_class != y_train[
    valid_index
  ]
)
}
)

mean(fold_errors)
}
)
)

#####
# MEILLEURS PARAMETRES
#####

best_xgb <- xgb_results %>%
  slice_min(
    cv_error,
    n = 1,
    with_ties = FALSE
  )

xgb_max_depth <- best_xgb$max_depth
xgb_nrounds <- best_xgb$nrounds
xgb_eta <- best_xgb$eta

cat(
  "Meilleur modèle XGBoost : max_depth =",

```

```

xgb_max_depth,
", nrounds =",
xgb_nrounds,
", eta =",
xgb_eta,
", erreur CV =",
round(best_xgb$cv_error, 4),
"\n"
)

```

Meilleur modèle XGBoost : max_depth = 5 , nrounds = 200 , eta = 0.1 , erreur CV = 0.7936

```
#meilleur param : max_depth = 5 , nrounds = 200 , eta = 0.1 , erreur CV = 0.7936
```

```
# Affichage des 10 meilleures combinaisons
```

```

xgb_results %>%
  arrange(cv_error) %>%
  head(10)

```

	max_depth	nrounds	eta	cv_error
1	5	200	0.10	0.7936059
2	6	200	0.05	0.7937122
3	6	150	0.10	0.7937125
4	5	200	0.20	0.7940331
5	6	300	0.10	0.7940336
6	5	300	0.20	0.7941403
7	4	200	0.20	0.7942493
8	5	150	0.10	0.7946785
9	6	300	0.20	0.7948934
10	5	300	0.10	0.7950000

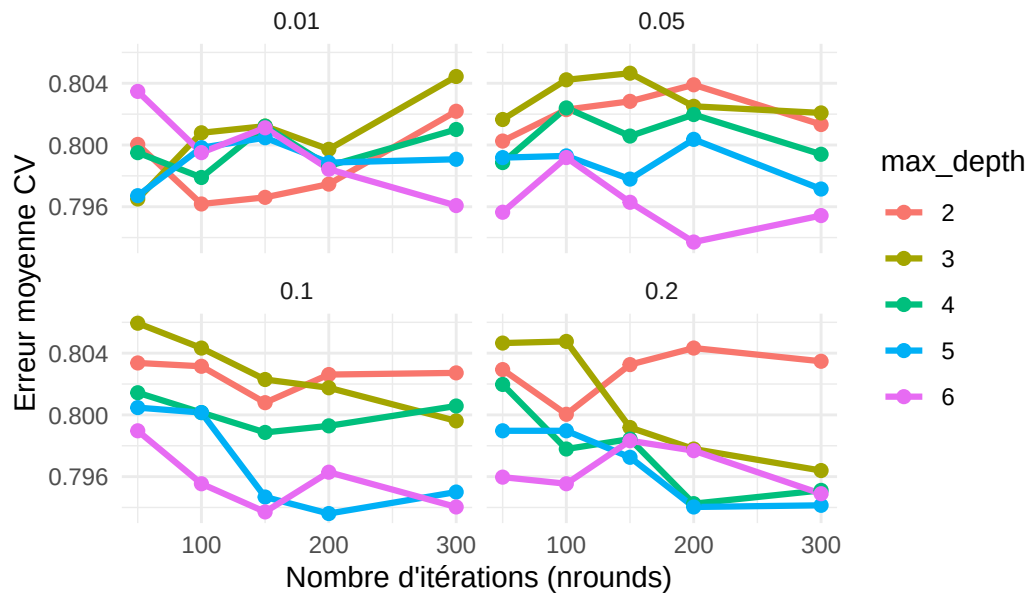
```
# Visualisation de l'erreur CV
```

```

ggplot(
  xgb_results,
  aes(
    x = nrounds,
    y = cv_error,
    color = factor(max_depth),
    group = max_depth
  )
) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2) +
  facet_wrap(~ eta) +
  labs(
    title = "Erreur de validation croisée - XGBoost",
    x = "Nombre d'itérations (nrounds)",
    y = "Erreur moyenne CV",
    color = "max_depth"
  ) +
  theme_minimal()

```

Erreur de validation croisée – XGBoost



```
xgb_max_depth
```

```
[1] 5
```

```
xgb_nrounds
```

```
[1] 200
```

```
xgb_eta
```

```
[1] 0.1
```

Avec les paramètres de RF et XGBoost, on peut passer au stacking.

6 Stacking

6.1 Préparation des folds

```
n_folds <- length(folds)

# Noms internes du stacking
rf_names <- paste0(
  "RF_C",
  seq_len(K)
)

xgb_names <- paste0(
  "XGB_C",
  seq_len(K)
)

oof_rf <- matrix(
  NA_real_,
  nrow = nrow(train_data),
  ncol = K,
  dimnames = list(
```

```

    NULL,
    rf_names
  )
)

oof_xgb <- matrix(
  NA_real_,
  nrow = nrow(train_data),
  ncol = K,
  dimnames = list(
    NULL,
    xgb_names
  )
)

```

On passe aux folds.

```

# Predictions out of fold pour rf et xg

for (i in seq_along(folds)) {

  valid <- folds[[i]]

  train <- setdiff(
    seq_len(nrow(train_data)),
    valid
  )

  train_fold <- train_data[
    train,
    ,
    drop = FALSE
  ]

  valid_fold <- train_data[
    valid,
    ,
    drop = FALSE
  ]

  # ----- RF -----

  rf <- ranger(
    Y ~ . - pm17B,
    data = train_fold,
    num.trees = rf_ntree,
    mtry = rf_mtry,
    min.node.size = rf_node,
    probability = TRUE,
    case.weights = train_fold$pm17B,
    seed = 123 + i
  )

  oof_rf[
    valid,
  ] <- predict(
    rf,

```

```

    valid_fold[
      ,
      rf_features,
      drop = FALSE
    ]
  )$predictions[
    ,
    classes,
    drop = FALSE
  ]

# ----- XGBoost -----

dtrain <- xgb.DMatrix(
  x_train[
    train,
    ,
    drop = FALSE
  ],
  label = y_train[
    train
  ],
  weight = w_train[
    train
  ]
)

dvalid <- xgb.DMatrix(
  x_train[
    valid,
    ,
    drop = FALSE
  ],
  label = y_train[
    valid
  ]
)

xgb <- xgb.train(
  params = list(
    objective = "multi:softprob",
    num_class = K,
    max_depth = xgb_max_depth,
    eta = xgb_eta,
    eval_metric = "mlogloss"
  ),
  data = dtrain,
  nrounds = xgb_nrounds,
  verbose = 0
)

oof_xgb[
  valid,
] <- matrix(
  predict(

```

```

    xgb,
    dvalid
  ),
  ncol = K,
  byrow = TRUE
)
}

```

6.2 Méta-modèle multinomial

```

meta_data <- data.frame(
  oof_rf,
  oof_xgb,
  Y = train_data$Y,
  check.names = FALSE
)

meta_model <- multinom(
  Y ~ .,
  data = meta_data,
  trace = FALSE
)

```

6.3 RF et XGBoost finaux

```

#####
# RF et XGBOOST finaux

# ----- RF -----

final_rf <- ranger(
  Y ~ . - pm17B,
  data = train_data,
  num.trees = rf_ntree,
  mtry = rf_mtry,
  min.node.size = rf_node,
  probability = TRUE,
  case.weights = w_train,
  importance = "permutation",
  seed = 999
)

pred_rf_test <- predict(
  final_rf,
  test_data[
    ,
    rf_features,
    drop = FALSE
  ]
)$predictions[
  ,
  classes,
  drop = FALSE
]

```

```
# ----- XGBoost -----

dtrain <- xgb.DMatrix(
  x_train,
  label = y_train,
  weight = w_train
)

dtest <- xgb.DMatrix(
  x_test,
  label = y_test,
  weight = w_test
)

final_xgb <- xgb.train(
  params = list(
    objective = "multi:softprob",
    num_class = K,
    max_depth = xgb_max_depth,
    eta = xgb_eta,
    eval_metric = "mlogloss"
  ),
  data = dtrain,
  nrounds = xgb_nrounds,
  verbose = 0
)

pred_xgb_test <- matrix(
  predict(
    final_xgb,
    dtest
  ),
  ncol = K,
  byrow = TRUE
)

colnames(pred_xgb_test) <- classes
```

6.4 Prédiction du stacking

```
colnames(pred_rf_test) <- rf_names
colnames(pred_xgb_test) <- xgb_names

meta_test <- data.frame(
  pred_rf_test,
  pred_xgb_test,
  check.names = FALSE
)

# Vérification importante
stopifnot(
  identical(
    names(meta_data)[
      names(meta_data) != "Y"
    ],
```

```

    names(meta_test)
  )
)

stacked_probs <- predict(
  meta_model,
  meta_test,
  type = "probs"
)

if (is.null(dim(stacked_probs))) {

  stacked_probs <- matrix(
    stacked_probs,
    ncol = K
  )
}

colnames(stacked_probs) <- classes

```

7 Premiers résultats

7.1 Classes prédites

```

rf_pred <- factor(
  classes[
    max.col(pred_rf_test)
  ],
  levels = classes
)

xgb_pred <- factor(
  classes[
    max.col(pred_xgb_test)
  ],
  levels = classes
)

stack_pred <- factor(
  classes[
    max.col(stacked_probs)
  ],
  levels = classes
)

```

7.2 Matrices de confusion

```

cm_rf <- confusionMatrix(
  rf_pred,
  test_data$Y,
  mode = "everything"
)

cm_xgb <- confusionMatrix(

```

```

    xgb_pred,
    test_data$Y,
    mode = "everything"
)

cm_stack <- confusionMatrix(
  stack_pred,
  test_data$Y,
  mode = "everything"
)

```

7.3 Comparaison des accuracies

```

accuracy_results <- data.frame(
  Modele = c(
    "Random Forest",
    "XGBoost",
    "Stacking"
  ),
  Accuracy = c(
    cm_rf$overall["Accuracy"],
    cm_xgb$overall["Accuracy"],
    cm_stack$overall["Accuracy"]
  )
)

print(accuracy_results)

```

	Modele	Accuracy
1	Random Forest	0.3960421
2	XGBoost	0.1971443
3	Stacking	0.4035571

7.4 Matrices de confusion pondérées

```

weighted_confusion <- function(
  pred,
  truth,
  weights
) {

  xtabs(
    weights ~ pred + truth,
    data = data.frame(
      pred = factor(
        pred,
        levels = classes
      ),
      truth = factor(
        truth,
        levels = classes
      ),
      weights = weights
    )
  )
}

```

```

}

weighted_cm_rf <- weighted_confusion(
  rf_pred,
  test_data$Y,
  w_test
)

weighted_cm_xgb <- weighted_confusion(
  xgb_pred,
  test_data$Y,
  w_test
)

weighted_cm_stack <- weighted_confusion(
  stack_pred,
  test_data$Y,
  w_test
)

```

7.5 Importance des variables

```

# Importance RF
rf_importance <- sort(
  final_rf$variable.importance,
  decreasing = TRUE
)

# Importance XGBoost
xgb_importance <- xgb.importance(
  model = final_xgb
)

```

7.6 Résultats finaux à comparer

```
cat("\n--- RF ---\n")
```

--- RF ---

```
print(cm_rf)
```

Confusion Matrix and Statistics

	Reference				
Prediction	13-	14	15	16	17+
13-	146	88	173	75	18
14	16	15	20	13	4
15	562	484	1245	556	128
16	59	58	113	175	43
17+	0	0	0	1	0

Overall Statistics

Accuracy : 0.396
95% CI : (0.3808, 0.4114)

No Information Rate : 0.3885
P-Value [Acc > NIR] : 0.169

Kappa : 0.085

McNemar's Test P-Value : <2e-16

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.18646	0.023256	0.8027	0.21341	0.0000000
Specificity	0.88969	0.984165	0.2913	0.91393	0.9997368
Pos Pred Value	0.29200	0.220588	0.4185	0.39062	0.0000000
Neg Pred Value	0.81758	0.839450	0.6991	0.81800	0.9516412
Precision	0.29200	0.220588	0.4185	0.39062	0.0000000
Recall	0.18646	0.023256	0.8027	0.21341	0.0000000
F1	0.22759	0.042076	0.5502	0.27603	NaN
Prevalence	0.19614	0.161573	0.3885	0.20541	0.0483467
Detection Rate	0.03657	0.003758	0.3119	0.04384	0.0000000
Detection Prevalence	0.12525	0.017034	0.7452	0.11222	0.0002505
Balanced Accuracy	0.53807	0.503710	0.5470	0.56367	0.4998684

```
cat("\n--- XGBoost ---\n")
```

--- XGBoost ---

```
print(cm_xgb)
```

Confusion Matrix and Statistics

	Reference				
Prediction	13-	14	15	16	17+
13-	160	126	303	150	34
14	146	141	324	171	49
15	149	97	288	169	41
16	176	133	297	154	25
17+	152	148	339	176	44

Overall Statistics

Accuracy : 0.1971
95% CI : (0.1849, 0.2098)
No Information Rate : 0.3885
P-Value [Acc > NIR] : 1

Kappa : 0.0029

McNemar's Test P-Value : <2e-16

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.20434	0.21860	0.18569	0.18780	0.22798
Specificity	0.80897	0.79385	0.81319	0.80107	0.78547
Pos Pred Value	0.20699	0.16968	0.38710	0.19618	0.05122
Neg Pred Value	0.80646	0.84056	0.61115	0.79233	0.95244
Precision	0.20699	0.16968	0.38710	0.19618	0.05122

Recall	0.20434	0.21860	0.18569	0.18780	0.22798
F1	0.20566	0.19106	0.25098	0.19190	0.08365
Prevalence	0.19614	0.16157	0.38853	0.20541	0.04835
Detection Rate	0.04008	0.03532	0.07214	0.03858	0.01102
Detection Prevalence	0.19364	0.20817	0.18637	0.19664	0.21518
Balanced Accuracy	0.50666	0.50622	0.49944	0.49444	0.50672

```
cat("\n--- Stacking ---\n")
```

--- Stacking ---

```
print(cm_stack)
```

Confusion Matrix and Statistics

	Reference					
Prediction	13-	14	15	16	17+	
13-	114	75	129	55	14	
14	0	0	0	0	0	
15	615	517	1323	591	144	
16	54	53	99	174	35	
17+	0	0	0	0	0	

Overall Statistics

Accuracy : 0.4036
 95% CI : (0.3883, 0.419)
 No Information Rate : 0.3885
 P-Value [Acc > NIR] : 0.02687

Kappa : 0.0812

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.14559	0.0000	0.8530	0.21220	0.00000
Specificity	0.91493	1.0000	0.2351	0.92402	1.00000
Pos Pred Value	0.29457	NaN	0.4147	0.41928	NaN
Neg Pred Value	0.81442	0.8384	0.7157	0.81940	0.95165
Precision	0.29457	NA	0.4147	0.41928	NA
Recall	0.14559	0.0000	0.8530	0.21220	0.00000
F1	0.19487	NA	0.5581	0.28178	NA
Prevalence	0.19614	0.1616	0.3885	0.20541	0.04835
Detection Rate	0.02856	0.0000	0.3314	0.04359	0.00000
Detection Prevalence	0.09694	0.0000	0.7991	0.10396	0.00000
Balanced Accuracy	0.53026	0.5000	0.5441	0.56811	0.50000

```
cat("\n--- Accuracy ---\n")
```

--- Accuracy ---

```
print(accuracy_results)
```

Modele Accuracy
 1 Random Forest 0.3960421

```
2      XGBoost 0.1971443
3      Stacking 0.4035571
```

```
cat("\n--- Importance RF ---\n")
```

```
--- Importance RF ---
```

```
print(rf_importance)
```

```

      B08A      B08B      Q10B1      Q10A1      Q09A1      Q09B1
0.0210739173 0.0187778987 0.0115860015 0.0103560390 0.0076848055 0.0076456189
      Q08C      Q05      Q06B      Q03      Q06A      Q04A
0.0053847145 0.0052252201 0.0050102503 0.0035789480 0.0034911982 0.0019615095
      Q08      Q04      Q04B
0.0013341998 0.0009577826 0.0006126623
```

```
cat("\n--- Importance XGBoost ---\n")
```

```
--- Importance XGBoost ---
```

```
print(xgb_importance)
```

	Feature <char>	Gain <num>	Cover <num>	Frequency <num>
1:	B08B1	0.056624839	0.025135205	0.028452626
2:	Q052	0.056588443	0.031324011	0.042655578
3:	Q031	0.053278131	0.035164043	0.046813680
4:	B08A1	0.049774457	0.022102478	0.027237899
5:	Q06B2	0.043880608	0.013238736	0.049757055
6:	Q10B18	0.038732091	0.018809114	0.019529060
7:	Q08C2	0.037466668	0.028400429	0.045598953
8:	Q10A17	0.029442766	0.010060095	0.026723977
9:	Q06A2	0.028332426	0.013341176	0.027284620
10:	B08A4	0.025138270	0.014845278	0.023687161
11:	B08A2	0.023655101	0.016549975	0.023874042
12:	Q04A1	0.021239171	0.014940506	0.019715941
13:	B08B4	0.020894167	0.014346439	0.022051953
14:	Q10B16	0.019451589	0.015823968	0.027938703
15:	B08B2	0.018965387	0.009073958	0.022659316
16:	Q10B14	0.018544291	0.021720285	0.017473369
17:	B08A3	0.017801688	0.011008502	0.021024108
18:	Q10A14	0.017577733	0.014953428	0.016258643
19:	Q09B13	0.016840024	0.025640285	0.020276584
20:	Q09B14	0.016778536	0.013090075	0.017426649
21:	Q04A2	0.016499931	0.009705686	0.017847131
22:	Q10A18	0.016232122	0.016365573	0.014950477
23:	B08B3	0.015979711	0.006914194	0.018174173
24:	B08A5	0.015724674	0.011768984	0.016445524
25:	Q09A16	0.015643150	0.038340472	0.012988226
26:	Q09B12	0.015642937	0.015411160	0.016819286
27:	Q10B15	0.015076143	0.009975296	0.016538965
28:	Q09B16	0.014285632	0.045280860	0.010698935
29:	Q09A15	0.014274551	0.016264834	0.014249673
30:	Q10A16	0.014163504	0.006571062	0.015604560
31:	B08B5	0.013685491	0.013823217	0.013922631
32:	Q06B3	0.013089274	0.014163303	0.016819286
33:	Q10A13	0.012811605	0.010581978	0.011399738

```

34:   Q082 0.012769118 0.014235721 0.016165203
35:  Q09A12 0.012219892 0.019208366 0.013642310
36:   Q042 0.011621893 0.011170980 0.009484209
37:  Q10A12 0.011503290 0.006421594 0.013735750
38:  Q10B17 0.011373998 0.013345252 0.012848066
39:  Q10B12 0.011252272 0.007908828 0.012801346
40:  Q10A11 0.011207008 0.030442331 0.011072697
41:  Q09A14 0.009172250 0.013242607 0.009624369
42:   Q084 0.009065738 0.016993295 0.010652215
43:  Q09A17 0.008908267 0.020275498 0.012521024
44:  Q10A15 0.007469095 0.015919986 0.010839095
45:  Q10B19 0.007446073 0.010457049 0.008129322
46:   Q04B2 0.007122305 0.012372823 0.009484209
47:   Q083 0.007094454 0.015894608 0.008736685
48:   Q06A3 0.006895852 0.012337452 0.008176042
49:   Q04B1 0.006215022 0.016808960 0.008876845
50:  Q10A19 0.005501131 0.009975518 0.007755560
51:  Q09B15 0.004460708 0.016379943 0.007054756
52:  Q10B13 0.004312661 0.011290995 0.007008036
53:   Q08C4 0.004308187 0.015388658 0.008035881
54:   Q04A3 0.003771585 0.014413620 0.006120351
55:  Q09B17 0.003767617 0.011298136 0.006727714
56:  Q10B11 0.003623921 0.012186251 0.006540833
57:   Q085 0.003140807 0.016481425 0.006634274
58:   Q08C5 0.002949916 0.007894268 0.003924500
59:  Q09A13 0.002537562 0.014270647 0.004999066
60:   Q04B4 0.002507750 0.009643180 0.004812185
61:   Q04B3 0.001841654 0.016162776 0.004438423
62:   Q08C3 0.001824865 0.022844627 0.006260512

```

```

Feature      Gain      Cover      Frequency
<char>      <num>      <num>      <num>

```

```
cat("\n--- Matrice pondérée RF ---\n")
```

```
--- Matrice pondérée RF ---
```

```
print(weighted_cm_rf)
```

```

      truth
pred    13-      14      15      16      17+
13-  148.810841  91.078820 171.529490  77.944175 16.963285
14   17.653443  13.902849  18.574371  13.169265  4.241537
15  564.800736 485.726520 1246.880043 545.967235 125.144090
16   57.594950  60.323810 111.875081 171.977059 44.008873
17+   0.000000   0.000000   0.000000   0.770407   0.000000

```

```
cat("\n--- Matrice pondérée XGBoost ---\n")
```

```
--- Matrice pondérée XGBoost ---
```

```
print(weighted_cm_xgb)
```

```

      truth
pred    13-      14      15      16      17+
13-  159.24317 123.43591 305.02602 147.42858 33.33885
14   147.28740 143.81941 322.24218 171.41685 46.94475
15   151.76853  97.16845 284.23539 172.07673 40.84471

```

```
16 177.70371 137.16140 293.87605 148.49773 25.38571
17+ 152.85716 149.44683 343.47935 170.40825 43.84376
```

```
cat("\n--- Matrice pondérée Stacking ---\n")
```

```
--- Matrice pondérée Stacking ---
```

```
print(weighted_cm_stack)
```

	truth				
pred	13-	14	15	16	17+
13-	117.26796	77.44093	128.56347	59.07140	13.78714
14	0.00000	0.00000	0.00000	0.00000	0.00000
15	618.92085	519.12243	1326.66576	581.25370	140.80973
16	52.67116	54.46864	93.62976	169.50304	35.76091
17+	0.00000	0.00000	0.00000	0.00000	0.00000